

Cloud handoff — #438, two server tests race on the process-global current repository

Written: 2026-08-25 · **By:** max (CLI session on Tom's box) · **For:** a cloud Claude Code session on `tom2025b/git-vista`

This replaced a handoff about #326 that was withdrawn — see `docs/handoffs/parked/README.md` for why. #438 is a better use of a session: it is pure Rust, it collides with nothing else in this batch, and **it failed CI this morning**, on the very branch these handoffs were written on.

The issue names one flaky test. **There are two.** The second is not in the issue and is the strongest evidence yet about the cause. Read "What the issue does not know" before anything else.

```
task_id: gv-438-parallel-test-race
issue: 438
milestone: –
repo: tom2025b/git-vista
base: main
branch: fix/438-parallel-test-race
sign_commits_as:
  name: Claude_Max
  email: 262510778+tom2025b@users.noreply.github.com
sign_artifacts_as: max
adr_number: 0083          # ASSIGNED. Do not pick "the next free" c
                        # 0078-0082 are taken by this batch; 0074
allowed_paths:
  - crates/git-vista-server/src/state.rs
  - crates/git-vista-server/src/recovery_center.rs
  - crates/git-vista-server/src/**          # the race may reach
  - docs/adr/
forbidden_paths:
  - design-docs/
  - ci/browser/**
  - crates/git-vista/src/**
  - crates/git-vista-protocol/src/**
  - handoff.md
merge_order: independent. Touches no file any other handoff in this
```

What the issue does not know

Issue #438 was filed about **one** test:

```
recovery_center::tests::a_stale_claimed_undo_is_refused_and_the_br  
panicked at crates/git-vista-server/src/recovery_center.rs:2036  
the refusal must say the offer changed, not something unrelated:  
This operation can no longer be recovered – its recovery point is
```

On 25 August, in GitHub Actions run 32834326578 (job 97759797651), that test failed alongside a second one the issue has never mentioned:

```
state::tests::selection_flow_carries_mode_and_gates_writes  
panicked at crates/git-vista-server/src/state.rs:1023
```

910 passed; 2 failed. A re-run of the same job on the same commit passed 912/912, and the full suite passes 912/912 locally on a 4-core box every time. So: same tree, same command, different outcome — and **two different tests, in two different files, failing in the same run.**

That pairing is the finding. One flaky test invites "fix the assertion"; two, in files that share nothing but a global, points at the global.

The global

```
crates/git-vista-server/src/state.rs:289:
```

```
static CURRENT: OnceLock<RwLock<Current>> = OnceLock::new();
```

One process-wide "current repository" — path, mode, and handle — written by `set_current_resolved`. Rust runs `#[test]` functions **on threads of one process**, so every test that selects a repository is writing that one cell, and every test that reads it can be reading another test's repository.

Both failing tests are consistent with exactly that:

- `selection_flow_carries_mode_and_gates_writes` (`state.rs:708`) is *about* selection and mode gating — it writes `CURRENT` and asserts on what it wrote.
- `a_stale_claimed_undo_is_refused_and_the_branch_is_left_alone` (`recovery_center.rs:1960`) got the **wrong refusal**: it expected "the offer changed" and got "recovery point no longer available". As #438 already says, that is not a wrong assertion — *two distinct refusal paths are racing, and under load the wrong one wins*. A recovery point resolved against a different repository than the one the test set up would produce precisely that.

Do not stop at that hypothesis because it is tidy. It is the strongest candidate and it is not proven. #438 itself offers two others worth eliminating rather than dismissing: a shared temp path or port, and a staleness decision made against a clock that other tests' load perturbs. Kill them with evidence, not with argument.

How to work this without guessing

Reproduce it first, and say how you did. A fix for an intermittent failure you never saw fail is a fix for nothing. Options, cheapest first:

- `cargo test -p git-vista-server -- --test-threads=N` for large N, in a loop.
- Run the two named tests together, repeatedly, with the rest of the suite as load.
- `--test-threads=1` should make it disappear entirely. **If it does not, the process-global hypothesis is wrong and you have learned something big** — say so loudly rather than proceeding.

Put the reproduction command and its observed failure rate in the PR body. "Ran it 200 times at 16 threads, saw 7 failures" is the deliverable; "should be fixed now" is not.

Then fix the cause, not the symptom. In rough order of preference:

1. **Give each test its own state** rather than sharing one global — a per-test handle threaded through, or a test-scoped override. This is the real fix and it is the one that also makes the next such test safe.
2. **Serialise only what must be serialised** — a mutex around the tests that genuinely need the global, rather than `--test-threads=1` for the whole suite. Slower is acceptable; a suite that lies is not.
3. `--test-threads=1` **in CI** is a last resort and a confession, not a fix. If you land it, say in the ADR that it is a workaround and what would replace it.

Do not "fix" this by relaxing an assertion. Both tests are asserting the right things. `a_stale_claimed_undo...` asserting that a stale claim is refused *for the stated reason* is the whole point — a refusal for the wrong reason is a different bug wearing the right outcome's clothes.

`allowed_paths` **deliberately includes all of** `crates/git-vista-server/src/**`. The race may live somewhere neither named file reaches, and a boundary that stops you from fixing the actual cause is a boundary that produces a symptom patch. Everything outside that crate is still forbidden: if the fix wants to reach the protocol crate or the frontend, stop and say so in the PR instead.

What you cannot run

The browser leg does not run in a cloud container — the kernel there reports `landlock_abi=-1`, the server refuses to start without its strict sandbox tier, and INV-13 gives it no degraded mode. Installing `bwrap` changes nothing; the missing capability is the kernel's. This is a Rust-test-harness task and should not need it. If your fix touches the server's startup path in a way a browser run would exercise, **say so explicitly in the PR body** — a session on Tom's box will run `ci/browser/run.sh` before merge.

The irony is worth naming: the flake you are fixing is one a cloud session would otherwise blame itself for. Every other handoff in this batch tells its session to re-run before believing these two tests are its own doing. After this lands, that paragraph should be deletable.

Acceptance

1. **The failure is reproduced before it is fixed**, with the command and an observed rate in the PR body.
2. The cause is named with evidence — a demonstration that the shared global (or whatever it actually turns out to be) is what the two tests are fighting over, not an argument that it probably is.
3. Both
`recovery_center::tests::a_stale_claimed_undo_is_refused_and_the_branch_is_left_alone` and
`state::tests::selection_flow_carries_mode_and_gates_writes`
pass under the reproduction command that previously failed them, over a run long enough to mean something. Say how long.
4. **No assertion is weakened.** If you changed one, that is a finding for its own section in the PR body, with the argument.

5. The fix is proved able to go red **two different ways** — remove the isolation, and weaken it (isolate one test but not the other, say). One `caught` verdict is not proof: a Git-Vista test survived one mutation and caught another on 2026-08-22, and either alone gives the wrong verdict. For a race, a mutation that "catches" only sometimes is not a catch — run it enough times to say so honestly.
 5. **ADR 0083** records how test isolation works in this crate from now on, so the next test that touches `CURRENT` is written safely rather than discovering this again. `docs/adr/README.md` index updated.
 7. **Update #438 itself** to name the second test — it is currently a one-test issue about a two-test problem.
 3. `cargo fmt --all`, `cargo clippy --all-targets -- -D warnings`, `cargo test --workspace` green.
 9. PR body says `Closes #438`. **Never delete the branch.**
-

Signed: max · 2026-08-25T07:44:00-04:00